

Claude Code – Complete Slash Command Reference

Every slash command in the Claude Code CLI, organized by category, with a usage example for each. Type `/` in the CLI to open the command menu.

Contents

1. [Session & Conversation Management](#)
 2. [Context Management](#)
 3. [Code Review & Quality](#)
 4. [Planning & Task Execution](#)
 5. [Background & Parallel Work](#)
 6. [Configuration & Settings](#)
 7. [Model, Effort & Performance](#)
 8. [Permissions & Security](#)
 9. [Skills, Plugins & Agents](#)
 10. [Memory & Project Setup](#)
 11. [Git & GitHub Integration](#)
 12. [Cloud, Web & Remote](#)
 13. [IDE & Platform Integration](#)
 14. [Account, Usage & Billing](#)
 15. [Diagnostics & Troubleshooting](#)
 16. [UI, Themes & Display](#)
 17. [Miscellaneous](#)
 18. [Removed / Deprecated](#)
 19. [Top 15 for Daily Use](#)
-

1. Session & Conversation Management

`/clear [name]`

Starts a new conversation with empty context. Pass a name to label the previous conversation so you can find it in the `/resume` picker.

Aliases: `/reset`, `/new`

```
/clear  
/clear auth-refactor-attempt-1
```

Use `/compact` instead if you want to free context but keep working on the same task.

```
/resume [session]
```

Resume a conversation by ID or name. With no argument, opens the session picker. Background sessions appear marked `bg`.

Alias: `/continue`

```
/resume  
/resume auth-refactor-attempt-1
```

```
/rename [name]
```

Renames the current session and shows the name on the prompt bar. Without a name, one is auto-generated from the conversation.

```
/rename payment-gateway-migration  
/rename
```

```
/branch [name]
```

Branches the conversation at this point so you can try a different direction without losing the current state. You switch into the branch; the original stays available via `/resume`.

```
/branch try-redis-instead
```

```
/rewind
```

Rewind the conversation and/or code to an earlier point, or summarize from a selected message. This is the checkpointing feature.

Aliases: `/checkpoint`, `/undo`

```
/rewind
```

```
/export [filename]
```

Exports the current conversation as plain text. With a filename it writes directly to that file; without one, it opens a dialog to copy to clipboard or save.

```
/export chat.txt  
/export ~/notes/debug-session.md  
/export
```

```
/copy [N]
```

Copies the last assistant response to the clipboard. Pass `N` to copy the Nth-latest response. When code blocks are present, an interactive picker appears — press `w` to write to a file instead of the clipboard (useful over SSH).

```
/copy  
/copy 2
```

```
/recap
```

Generates a one-line summary of the current session on demand.

```
/recap
```

```
/btw [question]
```

Ask a side question without adding it to the conversation. Run with no question to see your most recent side answer.

```
/btw what does EACCES mean again?  
/btw
```

```
/exit
```

Exits the CLI. In an attached background session this detaches instead, and the session keeps running.

Alias: `/quit`

```
/exit
```

```
/goal [condition|clear]
```

Sets a goal — Claude keeps working across turns until the condition is met or judged impossible.

```
/goal all tests in the auth module pass  
/goal  
/goal clear
```

2. Context Management

```
/context [all]
```

Visualizes current context usage as a colored grid, with optimization suggestions for context-heavy tools and memory bloat. Pass `all` to expand the per-item breakdown in fullscreen mode.

```
/context  
/context all
```

```
/compact [instructions]
```

Frees context by summarizing the conversation so far. Optionally pass focus instructions for the summary.

```
/compact  
/compact keep the database schema decisions and drop the CSS debugging
```

```
/autocompact [auto|<tokens>]
```

Sets how full the context window gets before Claude Code compacts automatically.
(v2.1.221+)

```
/autocompact 500k
/autocompact auto
/autocompact
```

3. Code Review & Quality

```
/code-review [level] [--fix] [--comment] [target]
```

Reviews the current diff — or a PR number, branch, or path you pass — for correctness bugs and cleanup opportunities.

Alias: `/review`

Option	Meaning
<code>low</code> <code>medium</code> <code>high</code> <code>xhigh</code> <code>max</code>	Effort levels
<code>ultra</code>	Deep cloud review
<code>--fix</code>	Apply the findings
<code>--comment</code>	Post findings as inline GitHub PR comments

```
/code-review
/code-review high
/code-review high --fix
/code-review 1234 --comment
/code-review ultra 1234
/code-review medium src/auth/
```

```
/review [...]
```

Alias of `/code-review`, taking the same levels and flags. With no level, it reuses the last one you typed.

```
/review  
/review 1234  
/review feature/checkout --fix
```

`/simplify [target]`

Runs four parallel review agents looking for reuse of existing helpers, simplification, efficiency, and correct abstraction level — then applies the fixes.

Since v2.1.154 this does **not** look for correctness bugs. Use `/code-review` for that.

```
/simplify  
/simplify src/components/  
/simplify 1234
```

`/security-review`

Analyzes the changes on your current branch for security vulnerabilities — injection, auth issues, data exposure. Requires an `origin` remote.

```
/security-review
```

`/ultrareview [PR or branch]`

Deep multi-agent review in a cloud sandbox. `/code-review ultra` is now the preferred form. Includes 3 free runs on Pro and Max, then requires usage credits.

```
/ultrareview  
/ultrareview 1234  
/ultrareview develop
```

`/diff`

Opens an interactive diff viewer for uncommitted changes and per-turn diffs.

Key	Action
← →	Switch between git diff and individual Claude turns
↑ ↓	Browse files
Enter	Open the selected file's diff
PageUp / PageDown	Scroll
Esc	Back to file list

```
/diff
```

4. Planning & Task Execution

```
/plan [description]
```

Enters plan mode directly. Pass a description to enter plan mode and start on that task immediately.

```
/plan  
/plan fix the auth bug  
/plan migrate the user table to include soft deletes
```

```
/batch <instruction>
```

Skill. Orchestrates large-scale changes in parallel: researches the codebase, decomposes the work into 5–30 independent units, presents a plan, then spawns one background subagent per unit in its own git worktree. Each subagent implements, tests, and opens a PR. Requires a git repository.

```
/batch migrate src/ from Solid to React  
/batch add TypeScript types to every file in lib/  
/batch replace all moment.js usage with date-fns
```

```
/loop [interval] [prompt]
```

Skill. Runs a prompt repeatedly while the session stays open. Omit the interval and Claude self-paces; omit the prompt and it runs an autonomous maintenance check or the prompt in

```
.claude/loop.md.
```

Alias: /proactive

```
/loop 5m check if the deploy finished  
/loop 30s tail the logs and tell me if any 500s appear  
/loop
```

```
/run
```

Skill. Launches and drives your project's app so you can see a change actually working, not just tests passing. (v2.1.145+)

```
/run
```

```
/verify
```

Skill. Confirms a change does what it should by building, running, and observing the app rather than relying on tests or type checks. (v2.1.145+)

```
/verify
```

```
/run-skill-generator
```

Skill. Writes a per-project skill that teaches /run and /verify how to build, launch, and drive your app from a clean environment.

```
/run-skill-generator
```

```
/deep-research <question>
```

Workflow. Fans out web searches, fetches and cross-checks sources, and synthesizes a cited report.


```
/deep-research what are the tradeoffs between Drizzle and Prisma in 2026
/deep-research current best practices for JWT refresh token rotation
```

```
/dataviz [request]
```

Skill. Design guidance for charts, graphs, and dashboards — picks the chart form for the data, assigns color by role, and validates the palette for colorblind safety and contrast. (v2.1.198+)

```
/dataviz build a revenue dashboard from the sales table
/dataviz
```

5. Background & Parallel Work

These four are easy to confuse. The difference is **where the work happens** and **where the result lands**:

Command	What it does	Where the result goes
<code>/background</code>	Detaches <i>this</i> session to run in the background	Frees your terminal
<code>/fork</code>	Copies the conversation into a <i>separate</i> background session	Stays separate
<code>/subtask</code>	Spawns a subagent that inherits the conversation	Returns into this conversation
<code>/branch</code>	Branches the conversation and switches you into it	You're inside it

```
/background [prompt]
```

Detaches the current session to run as a background agent, freeing the terminal. Pass a prompt to send one last instruction before detaching. Monitor with `claude agents`.

Alias: `/bg`

```
/background  
/bg finish the migration and open a PR when tests pass
```

```
/fork [prompt]
```

Copies the conversation into a new background session while you keep working here.
(v2.1.212+)

```
/fork  
/fork try solving this with a worker thread instead
```

```
/subtask <task>
```

Spawns a forked subagent that inherits the full conversation and works in the background. Its result returns to this conversation when it finishes. (v2.1.212+)

```
/subtask write integration tests for the payment webhook  
/subtask update the README to match the new API
```

```
/tasks
```

View and manage background work in the current session, including finished subagents.

Alias: /bashes

```
/tasks
```

```
/stop
```

Stops the current background session. Only available while attached to one. The transcript and any worktree are kept.

```
/stop
```

```
/workflows
```

Opens the workflow progress view to watch, pause, resume, or save running and completed workflows.

```
/workflows
```

```
/list-agents
```

Lists the subagents and other Claude Code sessions Claude can message. (v2.1.224+, cross-session messaging only)

Alias: /peers

```
/list-agents
```

6. Configuration & Settings

```
/config [key=value ...]
```

Opens the Settings interface. Since v2.1.181 you can set values directly; since v2.1.182 shorthand keys work too.

Alias: /settings

```
/config  
/config thinking=false  
/config theme=dark  
/config model=sonnet  
/config theme=dark model=opus  
/config --help
```

```
/status
```

Opens Settings on the Status tab — version, model, account, connectivity, and session kind. Works even while Claude is responding.

```
/status
```

`/keybindings`

Opens your keyboard shortcuts file.

```
/keybindings
```

`/statusline`

Configures the status line. Describe what you want, or run bare to auto-configure from your shell prompt.

```
/statusline  
/statusline show the git branch and current model
```

`/terminal-setup`

Configures terminal keybindings for Shift+Enter and similar shortcuts. Only visible in terminals that need it (VS Code, Cursor, Alacritty, Zed).

```
/terminal-setup
```

`/hooks`

View hook configurations for tool events.

```
/hooks
```

`/import [codex|gemini] [--dry-run] [--yes]`

Brings configuration from OpenAI Codex or Google Gemini CLI into Claude Code — instruction files, MCP servers, commands, subagents, and skills. (v2.1.213+)

```
/import  
/import codex --dry-run  
/import gemini --yes
```

7. Model, Effort & Performance

`/model [model]`

Switches the model and saves it as your default for new sessions. With no argument, opens a picker — press `s` on a row to switch for the current session only.

```
/model
/model opus
/model sonnet
/model haiku
```

`/effort [level|auto]`

Sets the model effort level. Takes effect immediately without waiting for the current response.

Level	Notes
<code>low</code>	Fastest, cheapest
<code>medium</code>	Balanced
<code>high</code>	Deeper reasoning
<code>xhigh</code>	Deeper still
<code>max</code>	Session-only
<code>ultracode</code>	Session-only
<code>auto</code>	Reset to model default

```
/effort
/effort high
/effort max
/effort auto
```

`/fast [on|off]`

Toggles fast mode. (v2.1.205+)

```
/fast on  
/fast off
```

```
/advisor [model|off]
```

Enables or disables the advisor tool, which consults a second model for guidance at key moments. Accepts `fable`, `opus`, `sonnet`, or a full model ID.

```
/advisor  
/advisor opus  
/advisor off
```

8. Permissions & Security

```
/permissions
```

Manages allow, ask, and deny rules for tool permissions. Opens an interactive dialog for viewing rules by scope, adding or removing them, managing working directories, and reviewing recent auto-mode denials.

Alias: `/allowed-tools`

```
/permissions
```

```
/fewer-permission-prompts
```

Skill. Scans your transcripts for common read-only Bash and MCP calls, then adds a prioritized allowlist to project `.claude/settings.json`.

```
/fewer-permission-prompts
```

```
/sandbox
```

Toggles sandbox mode (supported platforms only).

```
/sandbox
```

```
/privacy-settings
```

View and update privacy settings. Pro and Max subscribers only.

```
/privacy-settings
```

9. Skills, Plugins & Agents

```
/skills
```

Lists available skills. Type to filter, press `t` to sort by token count, `Space` to cycle a skill's visibility, `Enter` to save.

```
/skills
```

```
/reload-skills
```

Re-scans skill and command directories so changes made on disk during the session become available without restarting. (v2.1.152+)

```
/reload-skills
```

```
/plugin [subcommand]
```

Manages plugins. Run bare for the menu, or pass a subcommand.

```
/plugin  
/plugin list  
/plugin install my-org/deploy-tools  
/plugin enable deploy-tools  
/plugin disable deploy-tools
```

```
/reload-plugins [--force]
```

Reloads active plugins to apply pending changes without restarting. Warns and skips if the reload would invalidate the prompt cache — `--force` overrides.

```
/reload-plugins  
/reload-plugins --force
```

```
/agents
```

As of v2.1.198, prints a reminder to ask Claude to create or manage subagents, or to edit `.claude/agents/` directly. Earlier versions open an interactive interface.

```
/agents
```

```
/mcp [reconnect <server>|enable|disable [<server>|all]]
```

Manages MCP server connections and OAuth authentication.

```
/mcp  
/mcp reconnect github  
/mcp disable slack  
/mcp enable all
```

```
/claude-api [migrate|managed-agents-onboard|prompt-audit]
```

Skill. Loads Claude API and Managed Agents reference material for your project's language. Also activates automatically when your code imports `anthropic` or `@anthropic-ai/sdk`.

```
/claude-api  
/claude-api migrate  
/claude-api managed-agents-onboard  
/claude-api prompt-audit
```

10. Memory & Project Setup

```
/init
```

Initializes the project with a `CLAUDE.md` guide. Set `CLAUDE_CODE_NEW_INIT=1` for an interactive flow that also covers skills, hooks, and personal memory files.

```
/init
```

Do this first in any new project. Without `CLAUDE.md`, Claude doesn't know your conventions.

```
/memory
```

Edit `CLAUDE.md` files, enable or disable auto memory, and view auto memory entries.

```
/memory
```

```
/add-dir <path>
```

Adds a working directory for file access during the current session. Type a partial path and press Tab to accept a suggestion.

```
/add-dir ../shared-lib  
/add-dir ~/projects/design-system
```

Most `.claude/` configuration is **not** discovered from the added directory.

```
/cd <path>
```

Moves the session to a new working directory, keeping the conversation and prompt cache.
(v2.1.169+)

```
/cd ../backend  
/cd ~/projects/api-server
```

11. Git & GitHub Integration

```
/install-github-app
```

Installs the Claude GitHub App for a repository, with an optional step for GitHub Actions workflows and secrets.

```
/install-github-app
```

```
/autofix-pr [prompt]
```

Spawns a cloud session that watches the current branch's PR and pushes fixes when CI fails or reviewers comment. Requires the `gh` CLI and Claude Code on the web access.

```
/autofix-pr  
/autofix-pr only fix lint and type errors  
/autofix-pr address review comments but don't touch the test files
```

```
/web-setup
```

Connects your GitHub account to Claude Code on the web using your local `gh` CLI credentials.

```
/web-setup
```

12. Cloud, Web & Remote

```
/remote-control
```

Makes this session available for Remote Control from `claude.ai`. Requires a `claude.ai` subscription.

Alias: `/rc`

```
/remote-control  
/rc
```

```
/teleport
```

Pulls a Claude Code on the web session into this terminal — opens a picker, then fetches the branch and conversation.

Alias: `/tp`

```
/teleport
```

`/schedule [description]`

Creates, updates, lists, or runs routines that execute in the cloud. Claude walks you through setup conversationally.

Alias: `/routines`

```
/schedule  
/schedule run the test suite every weekday at 9am and report failures  
/schedule list my routines
```

`/remote-env`

Chooses the default environment for cloud agents.

```
/remote-env
```

`/desktop`

Continues the current session in the Claude Code Desktop app. Requires macOS or x64 Windows and a Claude subscription.

Alias: `/app`

```
/desktop
```

`/mobile`

Shows a QR code to download the Claude mobile app.

Aliases: `/ios`, `/android`

```
/mobile
```

13. IDE & Platform Integration

```
/ide
```

Manages IDE integrations and shows status.

```
/ide
```

```
/chrome
```

Configures Claude in Chrome settings.

```
/chrome
```

```
/install-slack-app
```

Installs the Claude Slack app; opens a browser to complete the OAuth flow.

```
/install-slack-app
```

```
/design-sync [hint]
```

Skill. Converts your repo's React design system and uploads it to Claude Design so generated designs use your real components. A first-time sync can take hours on a large repo.

```
/design-sync
```

```
/design-sync Acme DS
```

```
/design-login
```

Authorizes design-system access for `/design-sync` with your claude.ai account.

```
/design-login
```

```
/setup-bedrock
```

Interactive wizard for Amazon Bedrock authentication, region, and model pins. Only visible when `CLAUDE_CODE_USE_BEDROCK=1`.

```
/setup-bedrock
```

```
/setup-vertex
```

Interactive wizard for Google Cloud's Agent Platform. Only visible when `CLAUDE_CODE_USE_VERTEX=1`.

```
/setup-vertex
```

14. Account, Usage & Billing

```
/login / /logout
```

Sign in to or out of your Anthropic account.

```
/login  
/logout
```

```
/usage
```

Shows session cost, plan usage limits, and activity stats. On Pro, Max, Team, and Enterprise it includes a breakdown by skill, subagent, plugin, and MCP server.

Aliases: `/cost`, `/stats`

```
/usage  
/cost  
/stats
```

`/usage-credits`

Configure usage credits, or request them from your admin, when you hit a limit.

`/usage-credits`

`/upgrade`

Opens the upgrade page in your browser to switch to a higher plan tier.

`/upgrade`

`/passes`

Share a free week of Claude Code with friends. Only visible if your account is eligible.

`/passes`

15. Diagnostics & Troubleshooting

`/doctor`

Skill. A setup checkup that diagnoses issues and can fix them:

- Installation health — duplicate installs, PATH problems, unparseable settings
- Unused skills, MCP servers, and plugins versus their context cost
- Slow hooks
- Newer versions on your release channel
- Deduplicates local `CLAUDE.md` against checked-in ones and trims derivable content

Reports findings first and asks before changing anything.

Alias: `/checkup`

`/doctor`

From the shell, for read-only diagnostics without starting a session:

```
claude doctor
```

```
/debug [description]
```

Skill. Enables debug logging for the current session and troubleshoots by reading the debug log. Logging starts from the point you run it unless you launched with `claude --debug`.

```
/debug  
/debug the file watcher stops firing after about a minute
```

```
/heapdump
```

Writes a heap snapshot and memory breakdown to `~/Desktop` for diagnosing high memory usage. Doesn't appear in the command menu — type it in full.

```
/heapdump
```

 Attach only the `-diagnostics.json` file when reporting. The `.heapsnapshot` contains your full conversation and credentials — don't share it.

```
/bug [report]
```

Reports a bug or shares your conversation. You choose how much history to include and confirm on a consent screen.

Alias: `/share`

```
/bug  
/bug the diff viewer crashes when I open a file with CRLF line endings
```

```
/feedback [report]
```

Sends product feedback about Claude Code — same dialog and consent step as `/bug`.

```
/feedback  
/feedback would love a way to pin skills per project
```

16. UI, Themes & Display

```
/theme
```

Changes the color theme — auto (matches terminal background), light/dark variants, colorblind-accessible (daltonized) themes, ANSI themes, and custom themes from `~/.claude/themes/`.

```
/theme
```

```
/color [color|default]
```

Sets the prompt bar color for the current session. Available: red, blue, green, yellow, purple, orange, pink, cyan.

```
/color  
/color cyan  
/color default
```

```
/tui [default|fullscreen]
```

Sets the terminal UI renderer and relaunches with your conversation intact. `fullscreen` enables the flicker-free alt-screen renderer.

```
/tui  
/tui fullscreen  
/tui default
```

```
/focus
```

Toggles the focus view — shows only your last prompt, a one-line tool-call summary with edit diffstats, and the final response. Fullscreen rendering only.

```
/focus
```

```
/scroll-speed
```

Adjusts mouse wheel scroll speed interactively, with a live preview ruler.

```
/scroll-speed
```

17. Miscellaneous

```
/help
```

Shows help and available commands.

```
/help
```

```
/powerup
```

Quick interactive lessons with animated demos for discovering features.

```
/powerup
```

```
/insights
```

Generates an HTML report analyzing your recent sessions on this machine — which projects you work in, how you use Claude Code, where things go wrong, and features to try. Not available in cloud sessions.

```
/insights
```

```
/team-onboarding
```

Generates a team onboarding guide from your last 30 days of sessions, commands, and MCP usage. Produces a markdown guide a teammate can paste as a first message. Pro, Max, Team, and Enterprise also get a share link.

```
/team-onboarding
```

`/release-notes`

Views the changelog in an interactive version picker.

```
/release-notes
```

`/voice [hold|tap|off]`

Toggles voice dictation or enables it in a specific mode. Requires a Claude.ai account.

```
/voice
/voice hold
/voice tap
/voice off
```

`/radio`

Opens Claude FM lo-fi radio in your browser, or prints the stream URL if no browser is available.

```
/radio
```

`/stickers`

Orders Claude Code stickers.

```
/stickers
```

18. Removed / Deprecated

Command	Status	Use instead
<code>/vim</code>	Removed in v2.1.92	<code>/config</code> → Editor mode

Command	Status	Use instead
<code>/pr-comments</code>	Removed in v2.1.91	Ask Claude directly to view PR comments
<code>/ultraplan</code>	Removed	<code>/plan</code>
<code>/ultrareview</code>	Now an alias	<code>/code-review ultra</code>

Top 15 for Daily Use

#	Command	When
1	<code>/init</code>	First thing in a new project
2	<code>/clear</code>	Starting a genuinely new topic
3	<code>/compact</code>	Context is filling up
4	<code>/context</code>	Check how much room is left
5	<code>/resume</code>	Return to an earlier conversation
6	<code>/plan</code>	Plan before a large change
7	<code>/review</code>	Review the current diff
8	<code>/diff</code>	Inspect what changed
9	<code>/rewind</code>	Undo a wrong direction
10	<code>/model</code>	Switch models
11	<code>/effort</code>	Adjust reasoning depth
12	<code>/permissions</code>	Control permission prompts
13	<code>/memory</code>	Edit CLAUDE.md
14	<code>/usage</code>	Check cost and limits

#	Command	When
15	<code>/doctor</code>	When something feels off

Practical Tips

1. **Run** `/init` **first**. Without `CLAUDE.md`, Claude works without knowing your project's conventions.
 2. `/clear` **vs** `/compact` — new topic → `/clear`; same task, full context → `/compact`.
 3. **Check** `/context` **regularly**. Past ~70%, compact before quality degrades.
 4. **Run** `/fewer-permission-prompts` **after your first week**. It cuts permission fatigue dramatically.
 5. **Use** `/rewind` **freely**. If Claude heads the wrong way, undo and re-prompt rather than trying to correct mid-stream.
 6. **Audit skills** — in `/skills`, press `t` to sort by token count and turn off what you don't use.
 7. **Never share** `/heapdump` **output**. The `.heapsnapshot` contains credentials.
-

Reference: code.claude.com/docs